

Vorlesung 12: Lebesgue-Konstante und Chebychev-Punkte

Approximation · Jackson · Chebychev-Polynome

Warum diese Vorlesung?

Diese Vorlesung gehört zum Interpolationsblock. Prüfungsrelevant ist nicht nur, eine Formel hinzuschreiben, sondern zu entscheiden, *welche Darstellung* des Interpolationspolynoms numerisch sinnvoll ist: Lagrange ist konzeptionell klar, Aitken–Neville wertet an einem Punkt aus, Newton erlaubt effizientes Hinzufügen neuer Stützstellen, und Chebychev/Splines kontrollieren Instabilität und Oszillation.

1 Lebesgue-Konstante und Quasioptimalität

Definition 4.18 – Lebesgue-Konstante

Für Stützstellen $a = x_0 < \dots < x_n = b$ und zugehörige Lagrange-Polynome ℓ_j heißt

$$\Lambda_n := \max_{x \in [a, b]} \sum_{j=0}^n |\ell_j(x)|$$

die Lebesgue-Konstante der Stützstellen. Sie misst die absolute Kondition der Polynominterpolation.

Satz 4.19 – Quasioptimalität

Für den Interpolationsoperator $I^{(n)}$ gilt sinngemäß

$$\|f - I^{(n)}[f]\|_{\infty} \leq (1 + \Lambda_n) \inf_{q \in \mathcal{P}_n} \|f - q\|_{\infty}.$$

Die Interpolation ist also nur so gut wie die beste Approximation, multipliziert mit dem Stabilitätsfaktor $1 + \Lambda_n$.

Achtung

Äquidistante Punkte haben eine stark wachsende Lebesgue-Konstante. Daher kann Interpolation hoher Ordnung trotz Weierstraßsatz schlecht sein.

2 Weierstraß, Stetigkeitsmodul und Jackson

Satz 4.20 – Weierstraß

Für jede stetige Funktion $f \in C^0([-1, 1])$ und jedes $\varepsilon > 0$ gibt es ein Polynom $\pi_n \in \mathcal{P}_n$ mit

$$\|f - \pi_n\|_{\infty} \leq \varepsilon.$$

Das ist ein Existenzsatz für gute Polynome, aber kein Stabilitätssatz für eine konkrete Interpolationsstrategie.

Definition 4.22 – Stetigkeitsmodul

Für $f \in C^0([a, b])$ und $h > 0$ ist

$$\omega(f; h) := \max_{\substack{x, x' \in [a, b] \\ |x - x'| \leq h}} |f(x) - f(x')|.$$

Der Stetigkeitsmodul misst Glattheit auch dann, wenn f nicht differenzierbar ist, z.B. bei $f(x) = |x|$ oder $f(x) = \sqrt{x}$.

Satz 4.23 – Jackson-Abschätzung

Es gibt eine universelle Konstante $C_J > 0$, sodass für $f \in C^0([a, b])$ gilt

$$\min_{v \in \mathcal{P}_n} \|f - v\|_\infty \leq C_J \omega\left(f; \frac{b-a}{n}\right).$$

Für glattere Funktionen gilt z.B. für $f \in C^r([-1, 1])$:

$$\min_{v \in \mathcal{P}_n} \|f - v\|_\infty \leq C_r n^{-r} \|f^{(r)}\|_\infty.$$

3 Chebychev-Punkte

Definition 4.8.3 – Chebychev-Punkte

Die $n + 1$ Chebychev-Punkte in $[-1, 1]$ sind

$$x_i^{(n+1)} = \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad i = 0, \dots, n.$$

Sie liegen dichter an den Intervallrändern und reduzieren dadurch Randoszillationen.

Satz 4.26 – Min-Max-Eigenschaft des Knotenpolynoms

Sind x_i die Chebychev-Punkte und

$$\omega_{n+1}(x) = \prod_{i=0}^n (x - x_i) = x^{n+1} + q_n(x), \quad q_n \in \mathcal{P}_n,$$

dann gilt für jedes monische Polynom $\pi_{n+1}(x) = x^{n+1} + \tilde{q}_n(x)$

$$\|\pi_{n+1}\|_\infty \geq \|\omega_{n+1}\|_\infty = 2^{-n},$$

mit Gleichheit genau für das Chebychev-Knotenpolynom.

4 Chebychev-Polynome

Definition 4.27

Die Chebychev-Polynome sind rekursiv definiert durch

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x), \quad k \geq 2.$$

Lemma 4.28 – zentrale Eigenschaften

Für T_n gilt:

- T_n hat ganzzahlige Koeffizienten und Leitkoeffizient 2^{n-1} für $n \geq 1$.
- T_n ist gerade für gerades n und ungerade für ungerades n .
- Für $x \in [-1, 1]$ gilt $T_n(x) = \cos(n \arccos x)$ und damit $|T_n(x)| \leq 1$.
- Die Nullstellen sind

$$x_j^{(n)} = \cos\left(\frac{2j-1}{2n}\pi\right), \quad j = 1, \dots, n.$$

- Die Extremalstellen sind die Clenshaw–Curtis-Punkte $\bar{x}_j^n = \cos(j\pi/n)$.

Runge mit Chebychev-Punkten

Für $f(x) = (1+x^2)^{-1}$ auf $[-5, 5]$ verwendet man skalierte Chebychev-Punkte

$$x_j = 5 \cos\left(\frac{2j+1}{2n+2}\pi\right).$$

Im Gegensatz zu äquidistanten Stützstellen konvergiert der Interpolationsfehler hier gegen 0. Die Punkte häufen sich an den Rändern, wo das Runge-Phänomen sonst besonders stark wäre.

5 Python-Code – Chebychev-Punkte und Lebesgue-Funktion

```
import numpy as np

def chebyshev_nodes(n, a=-1.0, b=1.0):
    j = np.arange(n + 1)
    x = np.cos((2*j + 1) * np.pi / (2*n + 2))
    return 0.5*(a+b) + 0.5*(b-a)*x

def lebesgue_values(nodes, grid):
    nodes = np.asarray(nodes)
    grid = np.asarray(grid)
    n = len(nodes) - 1
    L = np.zeros_like(grid, dtype=float)
    for j in range(n + 1):
        ell = np.ones_like(grid, dtype=float)
        for k in range(n + 1):
            if k != j:
                ell *= (grid - nodes[k]) / (nodes[j] - nodes[k])
        L += np.abs(ell)
    return L
```

Prüfungscheckliste – Vorlesung 12

- | | | |
|--------------------------|---|-------------------------------|
| ☐ | Lebesgue-Konstante definieren und als Konditionsmaß erklären. | → <i>Def. 4.18</i> |
| <hr/> | | |
| ☐ | Quasioptimalitätsabschätzung interpretieren. | → <i>Satz 4.19</i> |
| <hr/> | | |
| ☐ | Stetigkeitsmodul und Jackson-Idee wiedergeben. | → <i>Def. 4.22, Satz 4.23</i> |
| <hr/> | | |
| ☐ | Chebyshev-Punkte korrekt formulieren. | → <i>Def. 4.8.3</i> |
| <hr/> | | |
| ☐ | Eigenschaften von T_n inklusive $\cos(n \arccos x)$ kennen. | → <i>Lemma 4.28</i> |
| <hr/> | | |
| ☐ | Runge-Phänomen mit Chebyshev-Punkten erklären. | → <i>Beispiel 4.30</i> |
| <hr/> | | |